

BEV-LaneDet: An Efficient 3D Lane Detection Based on Virtual Camera via Key-Points

Ruihao Wang^{1,2}, Jian Qin^{*1}, Kaiying Li¹, Yaochen Li², Dong Cao¹, Jintao Xu¹

¹HAOMO.AI Technology Co., Ltd.

²Xi'an Jiaotong University

{wangruihao, qinjian, likaiying, caodong, xujintao}@haomo.ai, yaochenli@mail.xjtu.edu.cn

Abstract

3D lane detection which plays a crucial role in vehicle routing, has recently been a rapidly developing topic in autonomous driving. Previous works struggle with practicality due to their complicated spatial transformations and inflexible representations of 3D lanes. Faced with the issues, our work proposes an efficient and robust monocular 3D lane detection called BEV-LaneDet with three main contributions. First, we introduce the Virtual Camera that unifies the in/extrinsic parameters of cameras mounted on different vehicles to guarantee the consistency of the spatial relationship among cameras. It can effectively promote the learning procedure due to the unified visual space. We secondly propose a simple but efficient 3D lane representation called Key-Points Representation. This module is more suitable to represent the complicated and diverse 3D lane structures. At last, we present a light-weight and chip-friendly spatial transformation module named Spatial Transformation Pyramid to transform multiscale front-view features into BEV features. Experimental results demonstrate that our work outperforms the state-of-the-art approaches in terms of F-Score, being 10.6% higher on the OpenLane dataset and 4.0% higher on the Apollo 3D synthetic dataset, with a speed of 185 FPS. Code is released at https://github.com/gigo-team/bev_lane_det.

1. Introduction

As one of the fundamental guarantees for autonomous driving, lane detection has recently received much attention from researchers. Robust lane detection in real-time is one of the foundations for advanced autonomous driving, which can provide substantial amounts of useful information for

Autonomous Driving Systems (ADS), vehicle self-control, localization, and map construction.

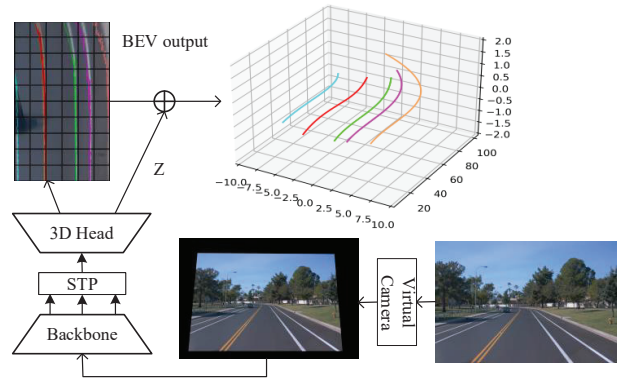


Figure 1. **End-to-end framework illustrated.** The original image in bottom-right is transformed to an input image by the *Virtual Camera* module. The input image is then encoded into front-view features by the backbone. Multiscale features from the backbone are put into the *Spatial Transformation Pyramid* (STP) to obtain BEV features. Given the BEV features, the 3D Head called *Key-Points Representation* generates BEV output and Z, the height of BEV lanes. At last, with the BEV output and Z, we can obtain 3D lanes.

2D lane detection methods have demonstrated remarkable performances [4, 18, 20, 22]. Moreover, their outputs are usually projected to the flat ground plane by Inverse Perspective Transformation (IPM) with the camera in/extrinsic parameters, and then curve fitting is performed to obtain the BEV lanes. However, the pipeline might cause other problems in the actual driving process [1, 20] for challenging situations like uphill and downhill.

In order to overcome these problems, more recent methods [3, 6, 8, 9, 14] have started to focus on the more complicated 3D lane perception domain. There are two significant challenges in 3D lane detection: an efficient spatial transformation module to obtain BEV features and a robust

*Corresponding author

This work was supported by the National Key Research and Development Project of New Generation Artificial Intelligence of China under Grant 2018AAA0102504.

representation for 3D lane structures. The acquisition of BEV features is heavily dependent on camera in/extrinsic parameters, and previous methods have chosen to incorporate camera in/extrinsic parameters into the network to obtain BEV features. Moreover, unlike obstacles on the road, lane structures are slender and diverse. These methods [3, 8, 9] carefully design the 3D anchor representation for lane structures with strong priors. However, they lack sufficient flexibility in some specific scenarios, as shown in Figure 5. Moreover, the anchor-free method [6] proposes a 3D lane representation based on the hypothesis that a line segment in each predefined tile is straight. This representation is complicated and inaccurate.

Towards the issues, we introduce BEV-LaneDet, an efficient and real-time pipeline that achieves 3D lane detection from a single image, as shown in Figure 1. Different from incorporating camera in/extrinsic parameters into the network to get BEV features, we establish a *Virtual Camera*, which is applied to images directly. The module unifies the in/extrinsic parameters of front-facing cameras in different vehicles by the homography method [2] based on BEV. This module guarantees the consistency of the spatial relationship of front-facing cameras in different vehicles and reduces variance in data distribution. Therefore, it can effectively promote the learning procedure due to the unified visual space. We also propose a *Key-Points Representation* as our 3D lane representation. We demonstrate that it is a simple but effective module to represent 3D lanes and is more expandable for complicated lane structures in some special scenarios. Moreover, the cost of computation and the chip's friendliness are also crucial factors in autonomous driving. Therefore, a light-weight and easy-to-deploy spatial transformation module based on MLP is our preference. Meanwhile, inspired by FPN [17], we present the *Spatial Transformation Pyramid*, which transforms multiscale front-view features to BEV and provides robust BEV features for 3D lane detection. In our experiments, we perform extensive studies to confirm that our BEV-LaneDet significantly outperforms the state-of-the-art PersFormer [3] in terms of F-Score, being 10.6% higher on the OpenLane real-world test set [3] and 4.0% higher on the Apollo simulation test set [9] with a speed of 185 FPS.

In summary, our main contributions are three-fold:

- 1) *Virtual Camera*, a novel preprocessing module to unify the in/extrinsic parameters of cameras, ensuring data distribution consistency.
- 2) *Key-Points Representation*, a simple but effective representation of 3D lane structures.
- 3) *Spatial Transformation Pyramid*, a light-weight and easy-to-deploy architecture based on MLP to realize transformation from multiscale front-view features to BEV. Experiments demonstrate that our BEV-LaneDet achieves the state-of-the-art performance compared to other 3D lane detection algorithms.

2. Related Work

2D Lane Detection. In recent years, there have been significant advancements in the field of 2D lane detection using deep neural networks (DNN). These works are divided into four categories according to pixel-wise segmentation, row-wise methods, anchor-based methods, and curve parameters. Some recent works [15, 20, 22, 32] consider 2D lane detection as a segmentation task based on pixel-wise, which the computing cost is expensive. Some methods [18, 24, 31] focus on the row-wise level to detect the 2D lanes. By setting the row anchors in the row direction and setting the grid cells in the column direction to model the 2D lanes on the image space, row-wise methods greatly improve the speed of inference. [27, 28] represent the lane structures with predefined anchors and regress offsets between sampled points and predefined anchor points to predict 2D lanes. These methods lack sufficient flexibility to accommodate complex lanes due to fixed anchors design. [7, 29] argue that the lane can be fitted by specific curve parameters on the 2D image space. So it is proposed that the 2D lane detection can be converted into the problem of curve parameter regression by detecting the starting point, ending point, and curve parameters. However, these methods need to combine camera intrinsic and extrinsic parameters for IPM projection to the ground in post-processing, which is based on the flat ground hypothesis. As mentioned in Section 1, this pipeline is not suitable for complicated road scenarios.

3D Lane Detection. In order to obtain more accurate road cognition results, many researchers have turned their attention to lane detection in 3D space. [3, 8, 9, 14] present remarkable results to prove the feasibility of using a CNN network for 3D lane detection in monocular images. 3D-LaneNet [8] firstly introduces a unified network for encoding 2D image information, spatial transformation and 3D lane detection in two path-ways: the image-view path encodes features from the 2D image, while the top-view path provides translation-invariant features for 3D lane detection. 3D-LaneNet+ [6] constructs the shape of lane segments in the predefined grid cells based on the straight segment hypothesis, which is complicated and might cause an error between the predicted segments and the actual lanes. Gen-LaneNet [9] proposes an extensible two-stage framework that separates the image segmentation subnetwork and the geometry encoding subnetwork. PersFormer [3] proposes a unified 2D and 3D lane detection framework and introduces Transformer [30] into the spatial transformation module to obtain more robust features. It also proposes a real-scene-based and large-scale annotated 3D lane dataset, OpenLane. These methods deploy intra-network feature maps IPM projection with camera in/extrinsic parameters, implicitly or explicitly. Different from applying IPM projection to features, we construct a *Virtual Camera* module, trying to project all images onto the view of a standard vir-

tual common camera. This approach ensures that the distribution of images is as consistent as possible. The distribution here includes the position, angle, and height of the camera from the ground. As for 3D lane representation, they carefully design 3D anchors with strong priors, which are complicated and lack expressiveness for some specific scenarios, as shown in Figure 5. We present our 3D lane representation, which is simple but robust, and it is more expandable for special scenarios.

Spatial Transformation. A vital module of 3D lane detection is the spatial transformation from front-view features to BEV features. The spatial transformation module [12] is a trainable module that is flexibly inserted into the CNN to implement the spatial transformation of the input features, and it is suitable for converting front-view features into BEV geometric features. There are four kinds of commonly used spatial transformation modules. IPM-based methods [8, 9, 26] rely heavily on the camera in/extrinsic parameters and ignore ground surface undulations and vehicle vibrations. The MLP-based methods [15, 21] are fixed spatial mapping, which are difficult to be integrated with the camera in/extrinsic parameters, resulting in poor performance. However, it is chip-friendly and rapid. Transformer-based spatial transformation modules [3, 16] are more robust, but they are not easy to deploy into autopilot chips due to the large amount of computation. The spatial transformation methods based on depth [11, 23] have a large amount of calculation and are thus not suitable for deployment. In combination with the *Virtual Camera* module, we can overcome the drawbacks of the MLP-based approaches and apply them to our method. We also introduce a feature pyramid inspired by FPN [17] to provide more robust BEV features.

3. Methodology

As shown in Figure 2, the whole network architecture consists of five parts: **1) Virtual Camera:** a preprocessing method for unifying camera intrinsic and extrinsic parameters; **2) Front-view Backbone:** a front-view features extractor; **3) Spatial Transformation Pyramid:** Projecting front-view features to BEV features; **4) Key-Points Representation:** a 3D head detector based on key-points; **5) Front-view Head:** a 2D lane detection head to provide auxiliary supervision.

Firstly, all in/extrinsic parameters of input images are transformed into unified in/extrinsic parameters through the *Virtual Camera*. This process ensures the consistency of the spatial relationship of front-facing cameras in different vehicles. We then use a feature extractor to extract the features of the front-view image. We carry out experiments with ResNet18 and ResNet34, respectively [10]. In order to promote the ability of the network to extract front-view features, a front-view lane detection head is added to serve

as auxiliary supervision. Inspired by [17], we design the *Spatial Transformation Pyramid*, a fast multiscale spatial transformation module based on [21]. This module is responsible for the transformation from front-view features to BEV features. Finally, we forecast the lane on the plane tangent to the local road surface P_{road} . P_{road} is the plane with $z = 0$ in the road ground coordinates $\mathcal{C}_{road} = (x, y, z)$. We divide the P_{road} into $s1 \times s2$ cells. Inspired by YOLO [25] and LaneNet [20], we predict the confidence, the embedding used for clustering, the offset from the cell center to the lane in the y direction of $\mathcal{C}_{road}$ and the height of each cell. In the inference, we use a fast clustering method to fuse the results of each branch to obtain 3D lanes.

3.1. Virtual Camera

The in/extrinsic parameters of different vehicles are various, which has a significant impact on the results of 3D lanes. Different from the methods that integrate the camera intrinsic and extrinsic parameters into the network features [3, 23], we realize a preprocessing method of quickly unifying the camera in/extrinsic parameters by establishing a *Virtual Camera* with standard in/extrinsic parameters.

We assume P_{road} to be the plane tangent to the local road surface. Because the 3D lane detection pays more attention to the plane P_{road} , we use the coplanarity of homography to project the image of the current camera to the view of the *Virtual Camera* through the homography matrix $H_{i,j}$. Therefore, the *Virtual Camera* achieves consistency of the spatial relationship of different cameras. As shown in Figure 3, the intrinsic parameters K_j and extrinsic parameters (R_j, T_j) of the *Virtual Camera* are fixed, which are derived from the mean value of the in/extrinsic parameters of the training dataset. In the training and inference stages, the homography $H_{i,j}$ is calculated according to the camera intrinsic parameters K_i and extrinsic parameters (R_i, T_i) provided by the current camera and the in/extrinsic parameters of the *Virtual Camera*. We refer [2] to calculate $H_{i,j}$. Firstly, we select four points $x^k = (x^k, y^k, 0)^T$ where $k = 1, 2, 3, 4$ on the BEV plane P_{road} . We then project them to the image of the current camera and the image of *Virtual Camera* respectively to obtain $u_i^k = (u_i^k, v_i^k, 1)^T$ and $u_j^k = (u_j^k, v_j^k, 1)^T$. Finally, $H_{i,j}$ is obtained by least square method, as shown in Eqn 1.

$$H_{i,j}u_i^k = u_j^k \quad (1)$$

During the inference, it is only necessary to perform the transformation, invoking *warpPerspective* if in OpenCV, with the already obtained $H_{i,j}$.

3.2. MLP Based Spatial Transformation Pyramid

The depth-based [11, 23] and Transformer-based methods [3, 16] are computationally expensive and unfriendly in

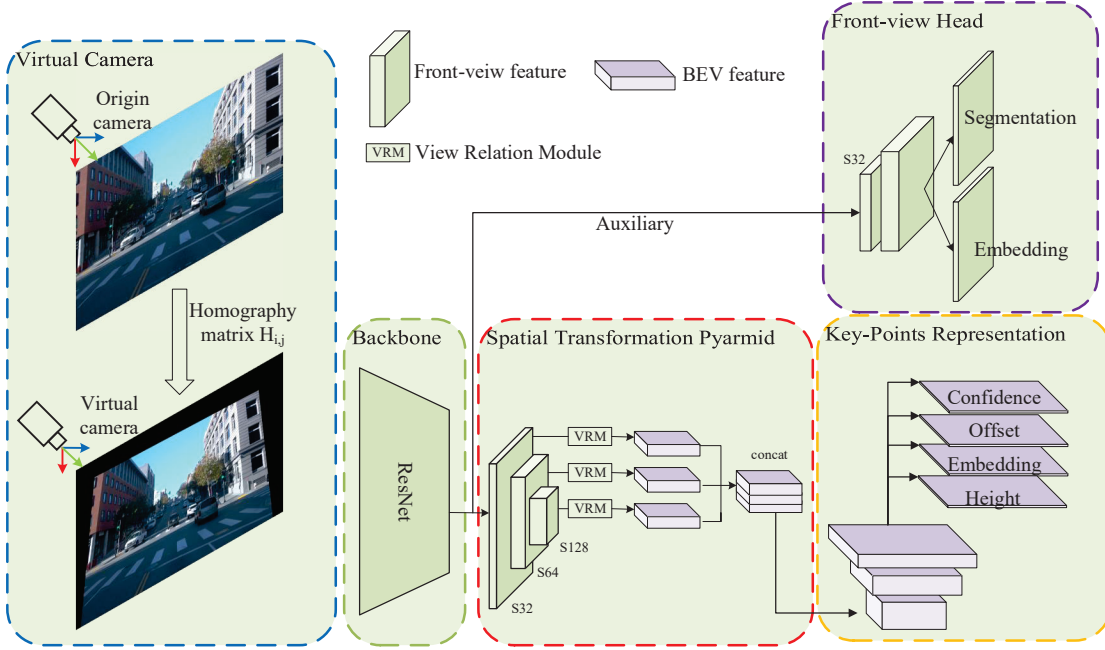


Figure 2. Our network structure consists of five parts: *Virtual Camera*, *Backbone*, *Spatial Transformation Pyramid*, *Key-Points Representation*, *Front-view Head*. $S32$: 32x downsampling of the input image.

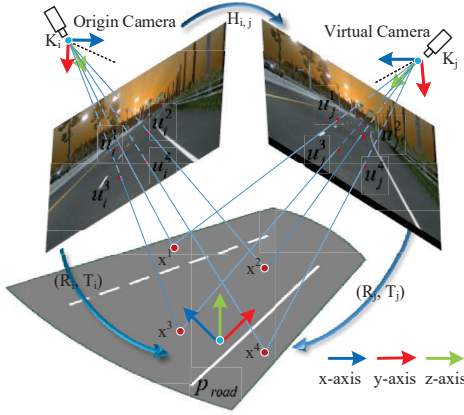


Figure 3. Schematic diagram of the *Virtual Camera*. The core aspect of the *Virtual Camera* is that the current camera and virtual camera are co-planar on P_{road} after Inverse Perspective Mapping (IPM).

deployment to autopilot chips. To address the issue, we introduce a light-weight and easy-to-deploy spatial transformation module referred to *View Relation Module* (VRM) [21] based on MLP. The module learns the relationships between any two pixel positions in the flattened front-view features and flattened BEV features using a view relation

module R . However, the VRM is a fixed mapping that ignores the variations brought by different camera parameters. Fortunately, the *Virtual Camera*, which unifies the in/extrinsic parameters of different cameras, makes up for this deficiency. The VRM is sensitive to the position of the front-view feature layer. We analyze the effect of different scales of front-view features in the VRM. Low resolution features are found to be more suitable for spatial transformation in the VRM through experiments. We consider that the low resolution features contain more global information. And since the MLP-based spatial transformation is a fixed mapping, the low-resolution features need fewer mapping parameters, which are easier to learn. Inspired by the FPN [17], we design a *Spatial Transformation Pyramid* based on VRM, as shown in the red box of Figure 2. By experimental comparison, we ultimately use the 1/64 resolution feature of the input image, $S64$ and the 1/32 resolution feature named $S32$ to be transformed, respectively, and then concatenate the results of both.

$$f_t[i] = \text{concat}(R_i^{S32}(f^{S32}[1], \dots, f^{S32}[HW^{S32}]), R_i^{S64}(f^{S64}[1], \dots, f^{S64}[HW^{S64}])) \quad (2)$$

where R_i^{S32} denotes the VRM of $S32$, $f_t[i]$ denotes pixel value of BEV features, HW^{S32} denotes the shape of $S32$, and $f^{S32}[j]$ denotes pixel value on $S32$.

Table 1. Comparison with other open-sourced 3D methods on the OpenLane. Our method achieves the best F-Score on the entire validation set and every scenario set.

Method	All	Up&Down	Curve	Extreme Weather	Night	Intersection	Merge&Split
3D-LaneNet [8]	40.2	37.7	43.2	43	39.3	29.3	36.5
Gen-LaneNet [9]	29.7	24.2	31.1	26.4	19.7	19.7	27.4
PersFormer [3]	47.8	42.4	52.8	48.7	37.9	37.9	44.6
Ours	58.4	48.7	63.1	53.4	53.4	50.3	53.7

Table 2. Comprehensive 3D lane evaluation under different metrics. Our Method outperforms previous 3D methods on the metrics of F-Score and speed.

Method	F-Score	X error near	X error far	Z error near	Z error far	Pytorch	TensorRT
3D-LaneNet [8]	40.2	0.278	0.823	0.159	0.714	-	-
Gen-LaneNet [9]	29.7	0.309	0.877	0.16	0.75	54FPS	-
PersFormer [3]	47.8	0.322	0.778	0.213	0.681	21FPS	-
Ours	58.4	0.309	0.659	0.244	0.631	102FPS	185FPS

3.3. Key-Points Representation

The representation of 3D lanes has a significant impact on the results of 3D lane detection. In this subsection, we propose a simple but robust representation to predict 3D lanes on BEV, referring to YOLO [25] and LaneNet [20]. As shown in Figure 4, we divide the BEV plane P_{road} , which is the plane with $z = 0$ in the road coordinates $C_{road} = (x, y, z)$ into $s1 \times s2$ cells. Each cell represents $x \times x$ (x defaults to $0.5m$). We directly predict the four heads with the same resolution, including the confidence, the embedding used for clustering, the offset from the cell center to the lane in y direction, and the average height of each cell. The size of the grid cell has a great influence on 3D lane prediction. An excessively small grid cell size affects the balance of positive and negative samples in the confidence branch. However, if the cell size is too large, the embedding of different lanes will overlap. Considering the sparsity of lane tasks, we recommend that the grid cell size be $0.5 \times 0.5 m^2$ through experiments. In training and inference, we predict the lanes of $(-10m, 10m)$ in the y direction and $(3m, 103m)$ in the x direction in the road ground coordinates $C_{road} = (x, y, z)$. Thus, four 200×40 resolution tensors, including confidence, embedding, offset and height are output from the 3D lane detection head. The confidence branch, embedding branch, and offset branch are merged to obtain the instance-level lanes under the BEV, as shown in Figure 4.

3.3.1 Confidence

Similar to YOLO [25], the confidence of lanes is a binary classification branch. Each pixel represents the confidence

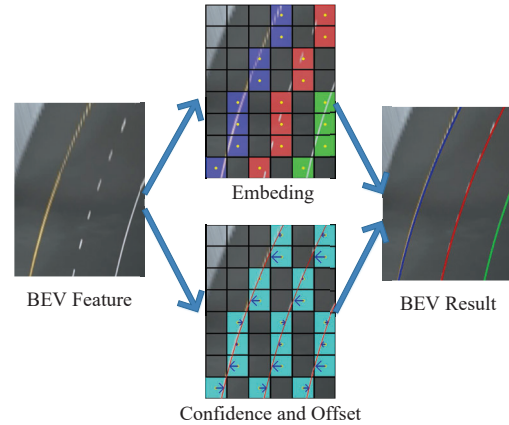


Figure 4. Schematic diagram of the *Key-Points Representation*. The BEV plane P_{road} is divided into $s1 \times s2$ grids. Each grid represents an area of $x \times x m^2$. BEV features are convolved to obtain three branches, including the embedding, confidence and offset. The three branches are merged to obtain the instance-level lanes on BEV.

of the cell. If there is a lane through the cell, the confidence score of the cell is set to one. Otherwise, the confidence score is set to zero. The confidence loss can be expressed by the Binary Cross Entropy loss.

$$L_{conf}^{3d} = \sum_i^{s1 \times s2} (\hat{p}_i \log p_i + (1 - \hat{p}_i) \log(1 - p_i)) \quad (3)$$

where p_i denotes the probability of the confidence predicted by the model, and $\hat{p}_i$ denotes the ground truth of confidence.

Table 3. There are results of different models on Apollo 3D Lane Synthetic. Compared with 3D-LaneNet [8], Gen-LaneNet [9], 3D-LaneNet(1/att) [13], Gen-LaneNet(1/att) [13], CLGO [19], Reconstruct from Top [14], and PersFormer [3], our model is the best on F-Score and X error.

Scene	Method	F-Score	X error near	X error far	Z error near	Z error far
Balanced Scence	3D-LaneNet [8]	86.4	0.068	0.477	0.015	0.202
	Gen-LaneNet [9]	88.1	0.061	0.496	0.012	0.214
	3D-LaneNet(1/att) [13]	91	0.082	0.439	0.011	0.242
	Gen-LaneNet(1/att) [13]	90.3	0.08	0.473	0.011	0.247
	CLGO [19]	91.9	0.061	0.361	0.029	0.25
	Reconstruct from Top [14]	91.9	0.049	0.387	0.008	0.213
	PersFormer [3]	92.9	0.054	0.356	0.01	0.234
	Ours	96.9	0.016	0.242	0.02	0.216
Rarely Observed	3D-LaneNet [8]	72	0.166	0.855	0.039	0.521
	Gen-LaneNet [9]	78	0.139	0.903	0.03	0.539
	3D-LaneNet(1/att) [13]	84.1	0.289	0.925	0.025	0.625
	Gen-LaneNet(1/att) [13]	81.7	0.283	0.915	0.028	0.653
	CLGo [19]	86.1	0.147	0.735	0.071	0.609
	Reconstruct from Top [14]	83.7	0.126	0.903	0.023	0.625
	PersFormer [3]	87.5	0.107	0.782	0.024	0.602
	Ours	97.6	0.031	0.594	0.04	0.556
Vivual Variants	3D-LaneNet [8]	72.5	0.115	0.601	0.032	0.23
	Gen-LaneNet [9]	85.3	0.074	0.538	0.015	0.232
	3D-laneNet(1/att) [13]	85.4	0.118	0.559	0.018	0.29
	Gen-LaneNet(1/att) [13]	86.8	0.104	0.544	0.016	0.294
	CLGo [19]	87.3	0.084	0.464	0.045	0.312
	Reconstruct from Top [14]	89.9	0.06	0.446	0.011	0.235
	PersFormer [3]	89.6	0.074	0.43	0.015	0.266
	Ours	95.0	0.027	0.32	0.031	0.256

Table 4. Ablation studies on OpenLane, all with 2D supervision. VC: Virtual Camera; STP: Spatial Transform Pyramid; KPR: Key-Points Representation; R34: ResNet34; R18: ResNet18.

Backbone	VC	STP	KPR	F-Score	X error	FPS
R34				51.2	0.37/0.79	-
R34	✓			54.5(+3.3)	0.32/0.69	-
R34		✓		53.2(+2.0)	0.37/0.79	-
R34			✓	53.5(+2.3)	0.37/0.76	-
R34		✓	✓	55.3(+4.1)	0.36/0.79	-
R34	✓		✓	56.7(+5.5)	0.31/0.69	-
R34	✓	✓	✓	58.4(+7.2)	0.31/0.66	185
R18	✓	✓	✓	57.8(+6.6)	0.32/0.70	272

3.3.2 Offset

Since the confidence branch does not accurately represent the location of lanes, the offset branch is responsible for predicting the precise offset from the cell center to the lane in the y direction of the road ground coordinates $\mathcal{C}_{road} = (x, y, z)$. As shown in Figure 4, the model predicts the offset Δy_i of each cell. The offset is normalized by the Sigmoid and subtracted by 0.5 so that the range of the offset is

Table 5. Impact of cell size and offset.

Cell size and offset	F-Score	X error near	X error far	GFLOPs
0.05m	43.2	0.3415	0.770	735.31
0.2m	55.7	0.321	0.701	89.42
0.5m	57.9	0.429	0.734	53.05
0.5m + offset	58.4	0.309	0.659	53.25
1m	56.8	0.607	0.856	47.97
1m + offset	57.7	0.317	0.671	48.08

$(-0.5, 0.5)$. The offset loss can be expressed by the MSE loss. Note that we only calculate offset for grid cells with a positive ground truth of confidence.

$$L_{offset}^{3d} = \sum_i^{s1 \times s2} 1_{obj}(\sigma(\Delta y_i) - \Delta \hat{y}_i)^2 \quad (4)$$

where 1_{obj} denotes whether the lane passes through this cell. Δy_i denotes offset from prediction, and $\Delta \hat{y}_i$ denotes offset from ground truth.

Table 6. The comparison of the different scales in front-view feature layers during spatial transformation. $S32$ represents 32x downsampling of the input image. $S32 + S64$ represents the concatenation of 64x downsampling and 32x downsampling of the input image.

Combination of scales	F-Score	X error near	X error far
S8	53.2	0.341	0.682
S16	57.1	0.340	0.696
S32	57.6	0.323	0.682
S64	56.7	0.322	0.697
S128	54.5	0.325	0.676
S32+S64	58.4	0.309	0.659
S32+S64+S128	58.3	0.315	0.656

3.3.3 Embedding

To distinguish the lane identity of each pixel in the confidence branch, we predict the embedding of each grid cell with reference to [5, 20]. In the training stage, the distance among cell embeddings belonging to the same lane is minimized, whereas the distance among cell embeddings belonging to different lanes is maximized. In the inference of the network, we use a fast unsupervised clustering post-processing method to predict the variable number of lanes. Unlike front-view lanes that usually converge at vanishing points, 3D lanes are more suitable for the embedding clustering loss function.

$$L_{embed}^{3d} = L_{var}^{3d} + L_{dist}^{3d} \quad (5)$$

where L_{var}^{3d} denotes the loss of minimizing the mean of cell embeddings belonging to the same lane. L_{dist}^{3d} denotes the loss of maximizing the variance of cell embeddings belonging to the different lanes.

3.3.4 Lane Height

Confidence, offset, and embedding can only predict the x and y of key points in the road ground coordinates $C_{road} = (x, y, z)$, and thus we present a height branch that is responsible for predicting the z of the key points. In the training phase of the network, we use the average height in a grid cell as ground truth. At the same time, only the grid cells with positive ground truth are counted in the loss.

$$L_{height}^{3d} = \sum_i^{s1 \times s2} 1_{obj}(h_i - \hat{h}_i)^2 \quad (6)$$

where h_i denotes the height of the grid cell predicted by the model, and $\hat{h}_i$ denotes the height of the grid cell from the ground truth.

3.3.5 Total loss

The total loss includes 3D lane losses and front-view lane losses. The front-view lane loss includes lane segmentation loss and lane embedding loss, referred to LaneNet [20].

$$L_{total} = \lambda_{conf}^{3d} L_{conf}^{3d} + \lambda_{embed}^{3d} L_{embed}^{3d} + \lambda_{offset}^{3d} L_{offset}^{3d} + \lambda_{height}^{3d} L_{height}^{3d} + \lambda_{seg}^{2d} L_{seg}^{2d} + \lambda_{embed}^{2d} L_{embed}^{2d} \quad (7)$$

where L_{seg}^{2d} denotes lane segmentation loss, and L_{embed}^{2d} denotes lane embedding loss in the front-view.

3.3.6 Inference

Given the outputs from KPR , we propose a fast unsupervised clustering method to obtain the instance-level lanes, which we refer to mean-shift from LaneNet [20]. We put the details of the algorithm in Appendix.

4. Experiments

In order to verify the performance of our work, our model is tested on the OpenLane real-world dataset [3] and the Apollo simulation dataset [9]. Compared with previous methods, including PersFormer [3], Reconstruct from Top [14], Gen-LaneNet [9], 3D-LaneNet [8], CLGO [19], etc., it is proven that our work can reach the state-of-the-art level in terms of F-Score and achieve competitive results in terms of X/Z error. The resolution of our input image is 576×1024 .

4.1. Evaluation Metrics and Implementation Details

On both 3D datasets, we adopt evaluation metrics coming from Gen-LaneNet [9], which include F-Score in various scenes and X/Z error in different areas.

4.2. Results on OpenLane

OpenLane contains 150,000 training frames and 40,000 test frames. In order to verify the performance of the model for every scene, the Up&Down case, Curve case, Extreme Weather case, Intersection case, Merge&Split case, and Night case are separated from the validation set. Table 1 shows the F-Score of the model in every scene. Our model trains 10 epochs in the training set and achieves the state-of-the-art performance for each scene. Table 2 shows the specific performance in terms of F-score and X/Z error for different methods. Our results are 10.6% higher than the state-of-the-art work [3] in terms of F-Score. The detailed visualization can be found in Appendix. Moreover, since the 3D ground truth of OpenLane is synthesized by LiDAR, our work does not show much advantage in X error. However, our work shows a great advantage with respect to X error for the Apollo dataset [9].

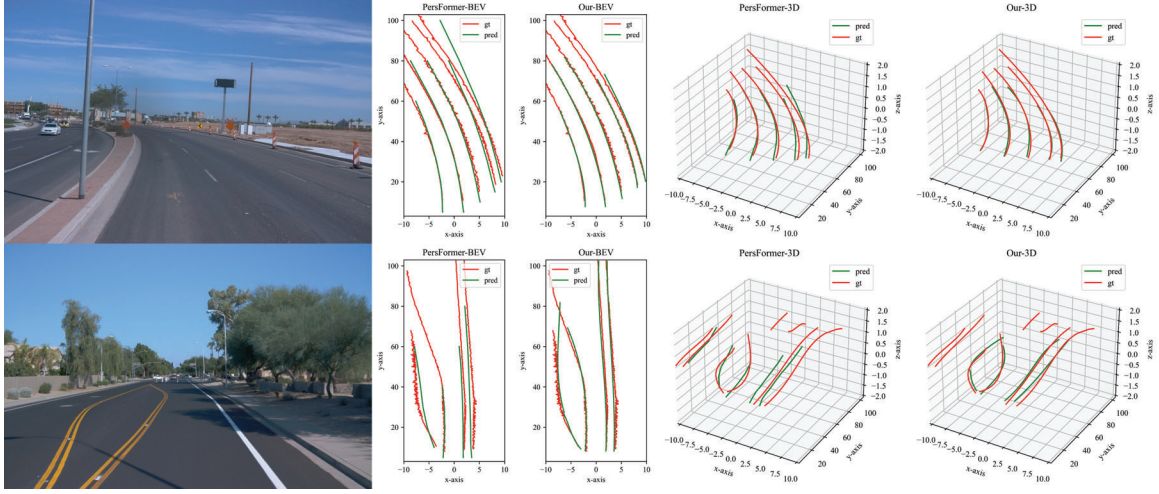


Figure 5. Qualitative results of PersFormer [3] and BEV-LaneDet on the Openlane dataset. The first column: the input images; The second column: the results of PersFormer in BEV; The third column: the results of our method in BEV; The fourth column: the results of PersFormer in 3D space; The fifth column: the results of our method in 3D space. The visualization results show that our method is more flexible and accurate.

4.3. Results on Apollo 3D synthetic

The Apollo dataset [9] includes 10,500 discrete frames of monocular RGB images and their corresponding 3D lanes ground truth, which is split into three scenes: balanced, rarely observed, and visual variation scenes. Each scene contains independent training sets and test sets. It is noted that Apollo does not provide specific extrinsic parameters of the camera. We calculate the extrinsic parameters of the camera through the height and pitch of the camera provided by the dataset. In Table 3, we provide a comparison between the previous works and our work. Our model has trained 80 epochs on these datasets. The F-Score and X error of our work both reach the state-of-the-art with respect to Apollo. However, since our work is more focused on the BEV plane, our work does not perform well in terms of Z error. We will improve this shortcoming in the future.

4.4. Ablation Study

The experiments in this section will be carried out on the OpenLane, and the evaluation metrics are still based on Gen-LaneNet [9]. Our baseline uses ResNet34 as the backbone, no *Virtual Camera*(VC), VRM [21] as the spatial transformation module, *Key-Points Representation* (KPR) with offset and a grid size of 0.2 for 3D lanes, and no 2D auxiliary supervision. We prove the effectiveness of our methods by adding three modules: VC, *Spatial Transformation Pyramid* (STP), and 2D auxiliary supervision. as shown in Table 4. Also, the table demonstrates the effect of changing the cell size from 0.2 to 0.5 in KPR, which we will discuss in detail in the following experiments. Meanwhile, we demonstrate the speed of different backbone running on

the Tesla-V100.

At the same time, in order to verify the effect of the offset and grid cell sizes on KPR, we add Table 5. In the experiments without offset, the F-Score keeps growing as the cell size grows. However, as the cell size increases, the X error also increases. When offset is added, the X error returns to the normal level for the large cell. Meanwhile, the larger the cell size is, the smaller FLOPs is. The experimental results demonstrate that the model achieves the best metrics when grid cell size is $0.5 \times 0.5 m^2$ with offset. Moreover, we explore the influence of the position of the front-view feature layer on the *View Relation Module* [21] in Table 6. Experiments show that the *Spatial Transformation Pyramid* achieves the best results when fusing the 64x down-sampling features and 32x down-sampling features of the original image.

5. Conclusions

In this paper, we propose BEV-LaneDet, a simple but effective 3D lane detection method. We present a *Virtual Camera* to guarantee the consistency of the spatial relationship of front-facing cameras in different vehicles, and we have proven its effectiveness through experiments. Moreover, we demonstrate experimentally that the *Spatial Transformation Pyramid*, which is a robust and chip-friendly module for spatial transformation, is effective. As shown in experiments, *Key-Points Representation* is a simple but effective module, which is more suitable to represent the diversity of lane structures. At last, we believe that our method can facilitate additional on-road 3D tasks.

References

- [1] Min Bai, Gellert Mattyus, Namdar Homayounfar, Shenlong Wang, Shrinidhi Kowshika Lakshmikanth, and Raquel Urtasun. Deep multi-sensor lane detection. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 3102–3109. IEEE, 2018. 1
- [2] Simon Baker, Ankur Datta, and Takeo Kanade. Parameterizing homographies. *Robotics Institute, Pittsburgh, PA, Tech. Rep. CMU-RI-TR-06-11*, 2006. 2, 3
- [3] Li Chen, Chonghao Sima, Yang Li, Zehan Zheng, Jiajie Xu, Xiangwei Geng, Hongyang Li, Conghui He, Jianping Shi, Yu Qiao, et al. Persformer: 3d lane detection via perspective transformer and the openlane benchmark. *arXiv preprint arXiv:2203.11089*, 2022. 1, 2, 3, 5, 6, 7, 8
- [4] Marius Cordts, Mohamed Omran, Sebastian Ramos, Timo Rehfeld, Markus Enzweiler, Rodrigo Benenson, Uwe Franke, Stefan Roth, and Bernt Schiele. The cityscapes dataset for semantic urban scene understanding. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3213–3223, 2016. 1
- [5] Bert De Brabandere, Davy Neven, and Luc Van Gool. Semantic instance segmentation with a discriminative loss function. *arXiv preprint arXiv:1708.02551*, 2017. 7
- [6] Netalee Efrat, Max Bluvstein, Shaul Oron, Dan Levi, Noa Garnett, and Bat El Shlomo. 3d-lanenet+: Anchor free lane detection using a semi-local representation. *arXiv preprint arXiv:2011.01535*, 2020. 1, 2
- [7] Zhengyang Feng, Shaohua Guo, Xin Tan, Ke Xu, Min Wang, and Lizhuang Ma. Rethinking efficient lane detection via curve modeling. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17062–17070, 2022. 2
- [8] Noa Garnett, Rafi Cohen, Tomer Pe’er, Roei Lahav, and Dan Levi. 3d-lanenet: end-to-end 3d multiple lane detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 2921–2930, 2019. 1, 2, 3, 5, 6, 7
- [9] Yuliang Guo, Guang Chen, Peitao Zhao, Weide Zhang, Jinghao Miao, Jingao Wang, and Tae Eun Choe. Gen-lanenet: A generalized and scalable approach for 3d lane detection. In *ECCV*, 2018. 1, 2, 3, 5, 6, 7, 8
- [10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016. 3
- [11] Junjie Huang, Guan Huang, Zheng Zhu, and Dalong Du. Bevdet: High-performance multi-camera 3d object detection in bird-eye-view. *arXiv preprint arXiv:2112.11790*, 2021. 3
- [12] Max Jaderberg, Karen Simonyan, Andrew Zisserman, et al. Spatial transformer networks. *Advances in neural information processing systems*, 28, 2015. 3
- [13] Yujie Jin, Xiangxuan Ren, Fengxiang Chen, and Weidong Zhang. Robust monocular 3d lane detection with dual attention. In *2021 IEEE International Conference on Image Processing (ICIP)*, pages 3348–3352. IEEE, 2021. 6
- [14] Chenguang Li, Jia Shi, Ya Wang, and Guangliang Cheng. Reconstruct from top view: A 3d lane detection approach based on geometry structure prior. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4370–4379, 2022. 1, 2, 6, 7
- [15] Qi Li, Yue Wang, Yilun Wang, and Hang Zhao. Hdmmapnet: A local semantic map learning and evaluation framework. *arXiv preprint arXiv:2107.06307*, 2021. 2, 3
- [16] Zhiqi Li, Wenhai Wang, Hongyang Li, Enze Xie, Chonghao Sima, Tong Lu, Qiao Yu, and Jifeng Dai. Bevformer: Learning bird’s-eye-view representation from multi-camera images via spatiotemporal transformers. *arXiv preprint arXiv:2203.17270*, 2022. 3
- [17] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2117–2125, 2017. 2, 3, 4
- [18] Lizhe Liu, Xiaohao Chen, Siyu Zhu, and Ping Tan. Condlanenet: a top-to-down lane detection framework based on conditional convolution. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 3773–3782, 2021. 1, 2
- [19] Ruijin Liu, Dapeng Chen, Tie Liu, Zhiliang Xiong, and Zejian Yuan. Learning to predict 3d lane shape and camera pose from a single image via geometry constraints. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 1765–1772, 2022. 6, 7
- [20] Davy Neven, Bert De Brabandere, Stamatios Georgoulis, Marc Proesmans, and Luc Van Gool. Towards end-to-end lane detection: an instance segmentation approach. In *2018 IEEE intelligent vehicles symposium (IV)*, pages 286–291. IEEE, 2018. 1, 2, 3, 5, 7
- [21] Bowen Pan, Jiankai Sun, Ho Yin Tiga Leung, Alex Andonian, and Bolei Zhou. Cross-view semantic segmentation for sensing surroundings. *IEEE Robotics and Automation Letters*, 5(3):4867–4873, 2020. 3, 4, 8
- [22] Xingang Pan, Jianping Shi, Ping Luo, Xiaogang Wang, and Xiaoou Tang. Spatial as deep: Spatial cnn for traffic scene understanding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018. 1, 2
- [23] Jonah Philion and Sanja Fidler. Lift, splat, shoot: Encoding images from arbitrary camera rigs by implicitly unprojecting to 3d. In *European Conference on Computer Vision*, pages 194–210. Springer, 2020. 3
- [24] Zequn Qin, Huanyu Wang, and Xi Li. Ultra fast structure-aware deep lane detection. In *European Conference on Computer Vision*, pages 276–291. Springer, 2020. 2
- [25] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 779–788, 2016. 3, 5
- [26] Lennart Reiher, Bastian Lampe, and Lutz Eckstein. A sim2real deep learning approach for the transformation of images from multiple vehicle-mounted cameras to a semantically segmented image in bird’s eye view. In *2020 IEEE 23rd International Conference on Intelligent Transportation Systems (ITSC)*, pages 1–7. IEEE, 2020. 3
- [27] Jinming Su, Chao Chen, Ke Zhang, Junfeng Luo, Xiaoming Wei, and Xiaolin Wei. Structure guided lane detection. *arXiv preprint arXiv:2105.05403*, 2021. 2

- [28] Lucas Tabelini, Rodrigo Berriel, Thiago M Paixao, Claudine Badue, Alberto F De Souza, and Thiago Oliveira-Santos. Keep your eyes on the lane: Real-time attention-guided lane detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 294–302, 2021. 2
- [29] Lucas Tabelini, Rodrigo Berriel, Thiago M Paixao, Claudine Badue, Alberto F De Souza, and Thiago Oliveira-Santos. Polylandenet: Lane estimation via deep polynomial regression. In *2020 25th International Conference on Pattern Recognition (ICPR)*, pages 6150–6156. IEEE, 2021. 2
- [30] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017. 2
- [31] Seungwoo Yoo, Hee Seok Lee, Heesoo Myeong, Sungrack Yun, Hyounghoo Park, Janghoon Cho, and Duck Hoon Kim. End-to-end lane marker detection via row-wise classification. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pages 1006–1007, 2020. 2
- [32] Tu Zheng, Hao Fang, Yi Zhang, Wenjian Tang, Zheng Yang, Haifeng Liu, and Deng Cai. Resa: Recurrent feature-shift aggregator for lane detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 3547–3554, 2021. 2